

A decorative graphic consisting of blue circuit lines with circular nodes, extending horizontally from the left and right sides of the central text box.

SHA3-256 BITCOIN MINER HARDWARE IMPLEMENTATION

BY DESMOND, JACK, THENUJA, FELIX



PROJECT DEMO STRUCTURE

- Initial project plan & aims
- Hardware Optimization Strategies
- Workflow overview & methodology
- Architecture Overview
- Group Member Contributions

INITIAL PROJECT PLAN

- Bitcoin Mining Basics:
 - Finding a nonce that makes a block's hash fall below a target threshold (ensuring finding a valid block is computationally difficult)
- Project Goals:
 - Develop a functional Bitcoin mining system on the Kira KV260 FPGA
 - Accelerate the HASH computations using the SHA3-256 algorithm
 - MAIN AIM: showcase the FPGA capability by achieving higher hash throughput and then analyse performance compared to an unoptimized version
- Why SHA3-256 (Keccak)?
 - Chosen for hardware-friendly design
 - Allow for ultra-fast, deeply pipelined hardware, and has up to 4x the performance compared to SHA2

MAIN APPROACH

- Focused on implementing the core hashing pipeline in hardware and interfacing it with a processor for control
- The design targets only the essential mining loop:
 - Insert image here
 - We ignore other block header details (Merkle root calculations, previous hash queries, versioning)
 - We use a fixed block header
- We only accelerate the Keccak hash computation for the nonce.
- This allows us to demonstrate the mining process:
 - Finding a nonce that produces a hash below a target

DESIGN

Keccak-256 algorithm in hardware:

- Implemented the full SHa3-256 hashing core in VHDL
- The design processes the fixed 80-byte block header plus a nonce and computes the 256-bit hash result on the FPGA

Keccak “sponge function”:

- Uses a 1600-bit internal state
- Our Design absorbs the input (block header + nonce) into the state then applies 24 rounds of the Keccak-f permutations (steps θ , ρ , π , χ , ι) to produce the hash output (Squeeze Phase)

DESIGN

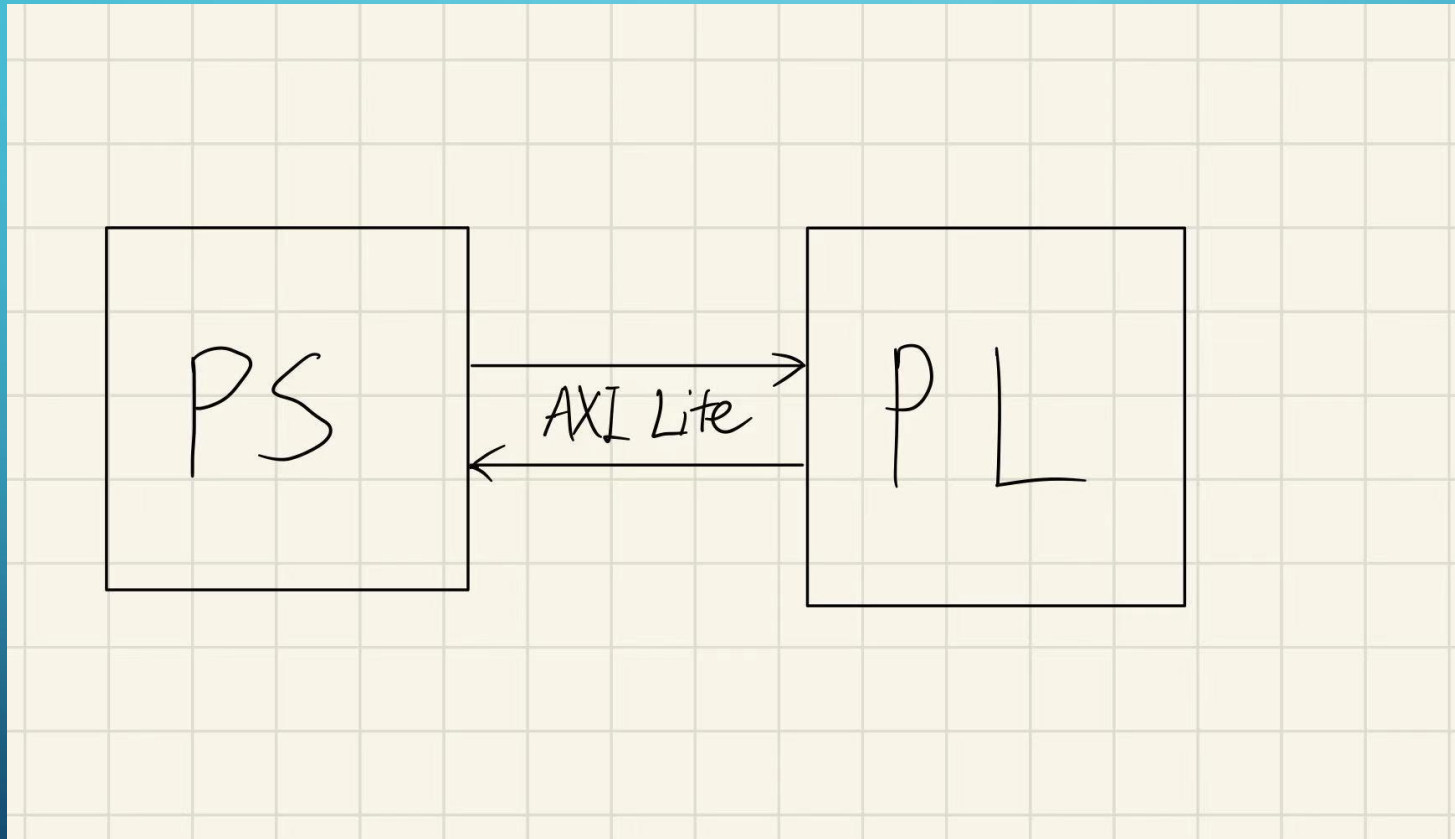
Pipeline and concurrency:

- Hashing Core internally pipelined
- Keccak round computation split into stages, allowing a new hash computation to be initiated every clock cycle for high throughput

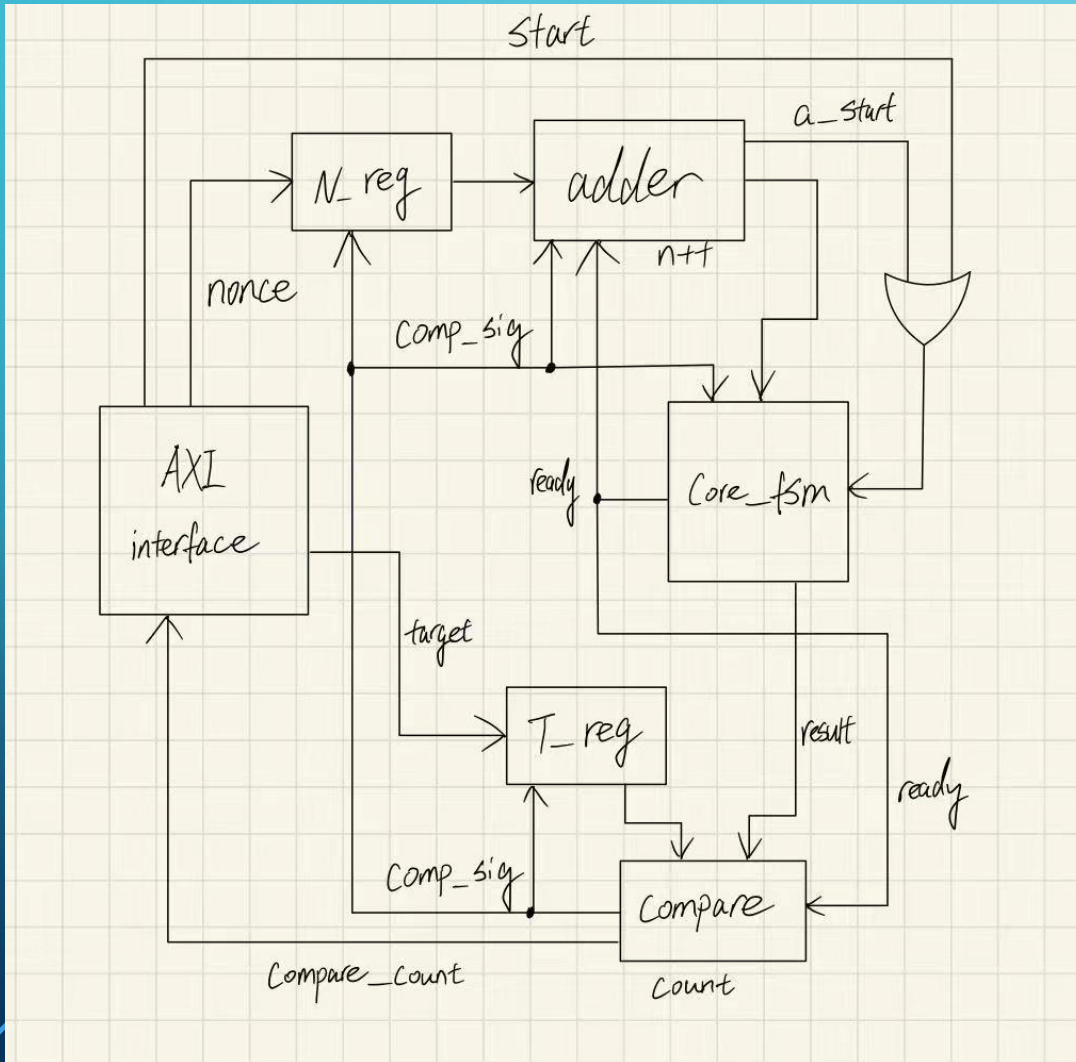
Design Optimizations:

- Fixed Input Size and known padding for simplified logic
- The core was verified in VHDL simulation against known hashes

OVERALL ARCHITECTURE & WORKFLOW

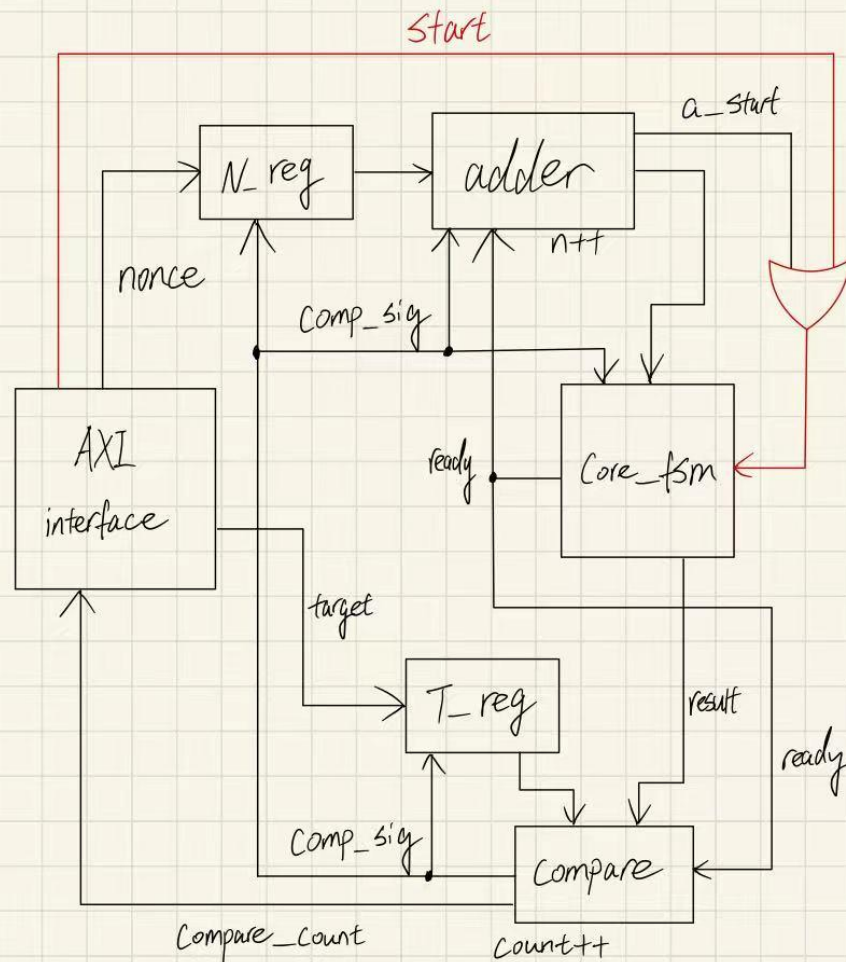


HARDWARE OVERVIEW



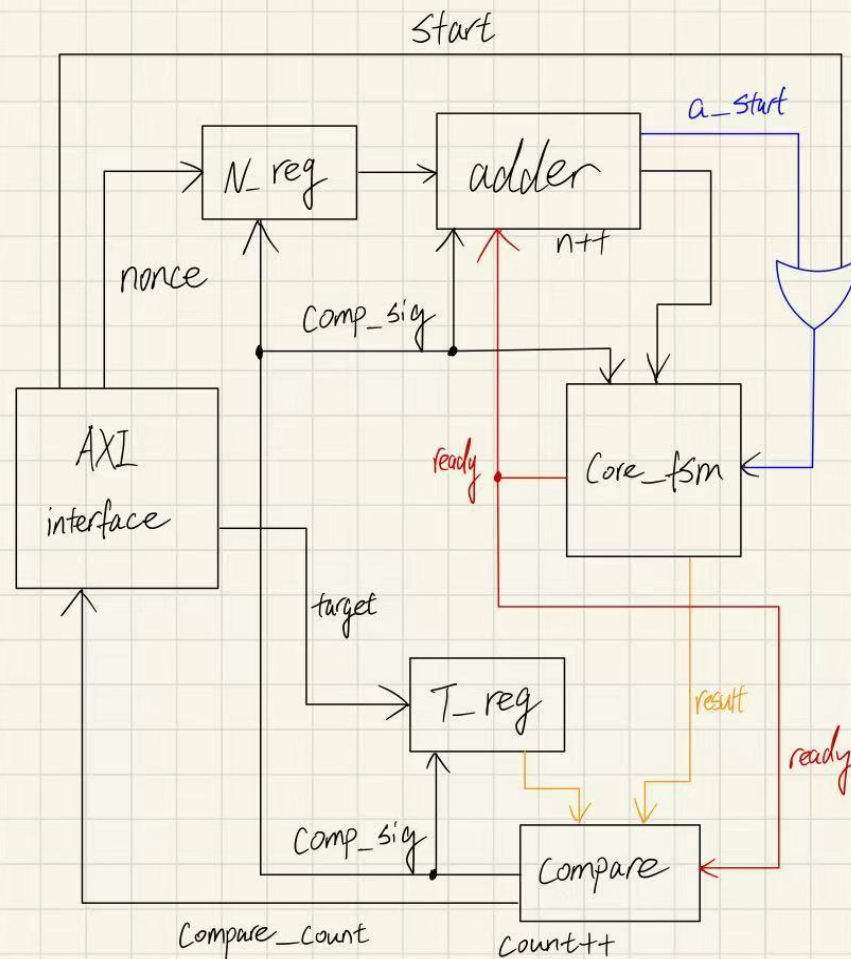
- AXI Interface
 - Communicates with the PS
 - Receives the nonce and target, and send back the result
- N_reg
 - Stores the nonce value
- Adder
 - Maintains an internal counter *n*
 - Adds *n* with the nonce to generate a new candidate for hashing
- Core_fsm
 - Controls the Keccak hashing process
 - Manages absorb, permute, and squeeze operations
- T_reg
 - Holds the target value used for comparison with the Keccak hash output
- Compare
 - Compares the hash result with the target
 - Also stores a value *n*. After each comparison, increase *n*.

SYSTEM START-UP



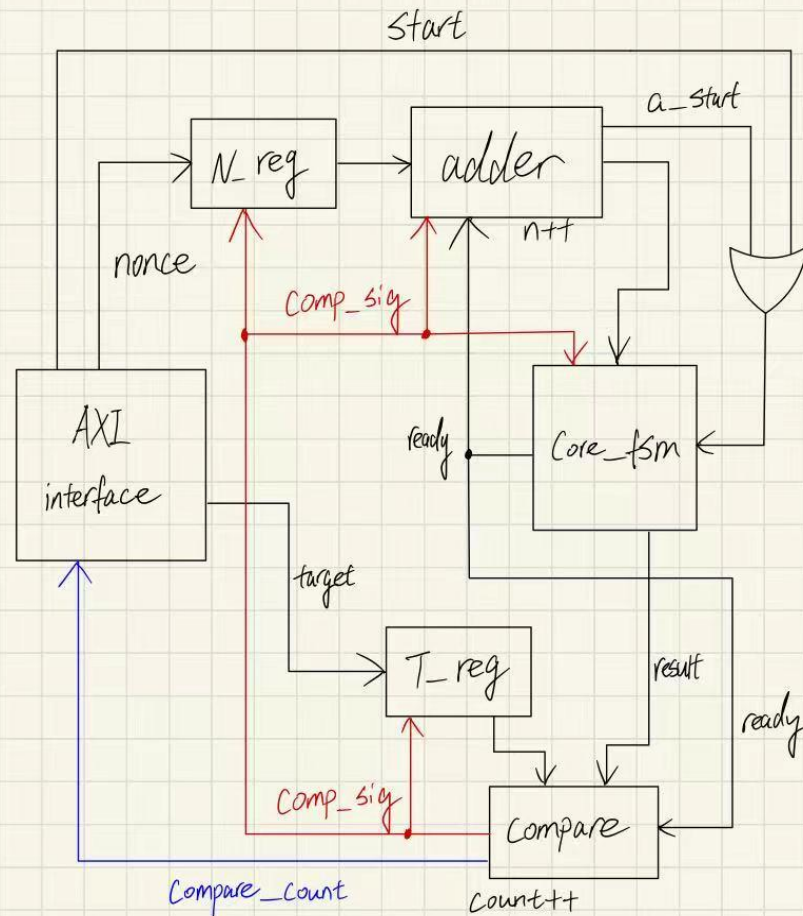
- The PS sends the nonce and target values via the AXI interface
- After the data is loaded, the PS asserts a control signal to notify the core_fsm to begin the hashing process
- The FSM will start fetching the nonce value from the adder

POST-HASH PROCESSING



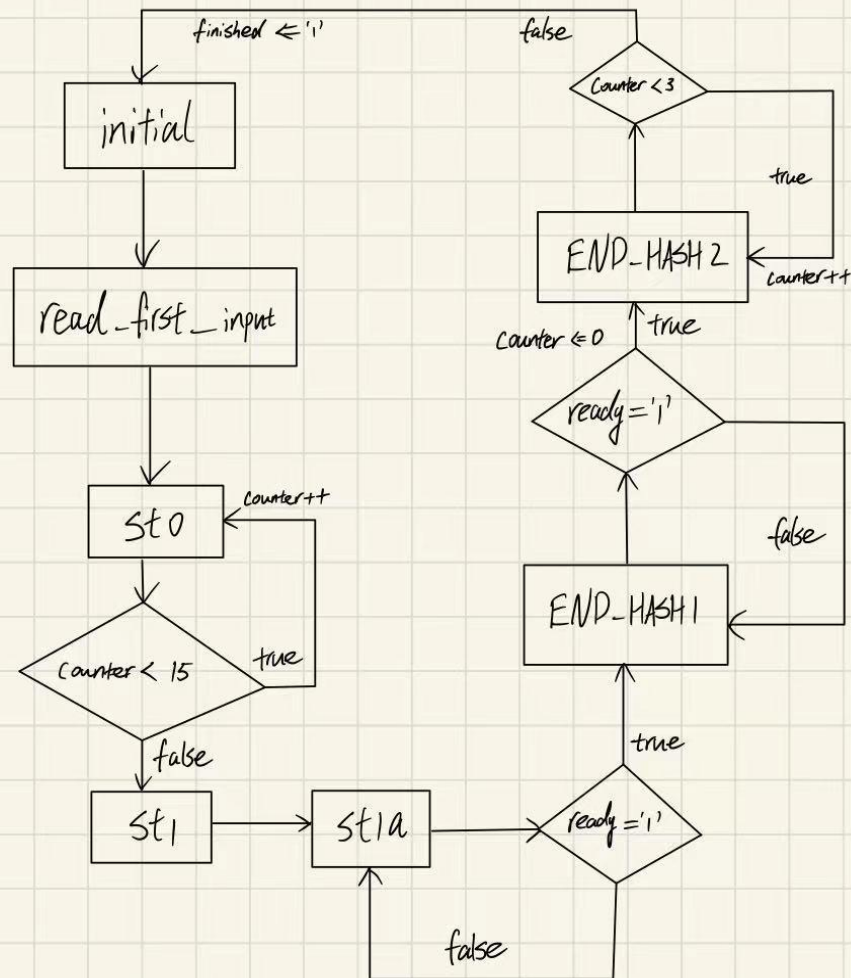
- Once the hashing process finish, the core_fsm generates hash result
- At the same time, it raises a ready signal to indicate the hash is ready to be compared
- The compare block detects the ready signal and performs actions
- The ready signal also triggers the adder

COMPLETION AND OUTPUT



- The compare component asserts the comp_sig signal, which resets all system components
- Sends the final value of n to the AXI interface for transmission to the PS
- The most significant bit of the transmitted n serves as a validation flag
 - MSB = 1: a valid hash result was found
 - MSB = 0: no valid result was found

CORE FSM



- Initial: Resets Keccak state, then starts input
- Read_first_input: Loads nonce, begins absorption
- St0: Absorbs chunks into Keccak
- St1: Starts permutation, moves to wait state (st1a)
- St1a: Waits for permutation to finish
- END_HASH1: Prepares for output, waits for valid data
- END_HASH2: Reads output. When done, resets FSM

CORE FSM

```
case st is
  when initial =>
    st <= read_first_input;
    finished <= '0';
    init <= '1';
  when read_first_input =>
    init <= '0';
    din <= nonce;
    absorb <= '1';
    counter <= 0;
    st <= st0;
  when st0 =>
    if counter < 15 then
      din <= (others => '0');
      absorb <= '1';
      counter <= counter + 1;
      st <= st0;
    else
      absorb <= '0';
      go <= '1';
      st <= st1;
    end if;
  when st1 =>
    go <= '0';
    st <= st1a;
```

```
when st1a =>
  if ready = '0' then
    st <= st1a;
  else
    st <= END_HASH1;
  end if;
when END_HASH1 =>
  if ready = '0' then
    st <= END_HASH1;
  else
    squeeze <= '1';
    counter <= 0;
    st <= END_HASH2;
  end if;
when END_HASH2 =>
  result <= dout;
  if counter < 3 then
    counter <= counter + 1;
    st <= END_HASH2;
  else
    squeeze <= '0';
    finished <= '1';
    st <= initial;
  end if;
end case;
```


KECCAK CORE

The Keccak algorithm FSM represents

- Pad and split message into r-bit blocks
- XOR block with first r bits of state
- Keccak-f permutation
 - XORing column parities
 - Rotates each lane by some offset
 - Rearrange lane positions
 - XOR and mix to add non-linearity
 - Salt with random constant
- SHA-3 proceeds with 24 rounds, each round includes 5 iterations of each step in Keccak-f.

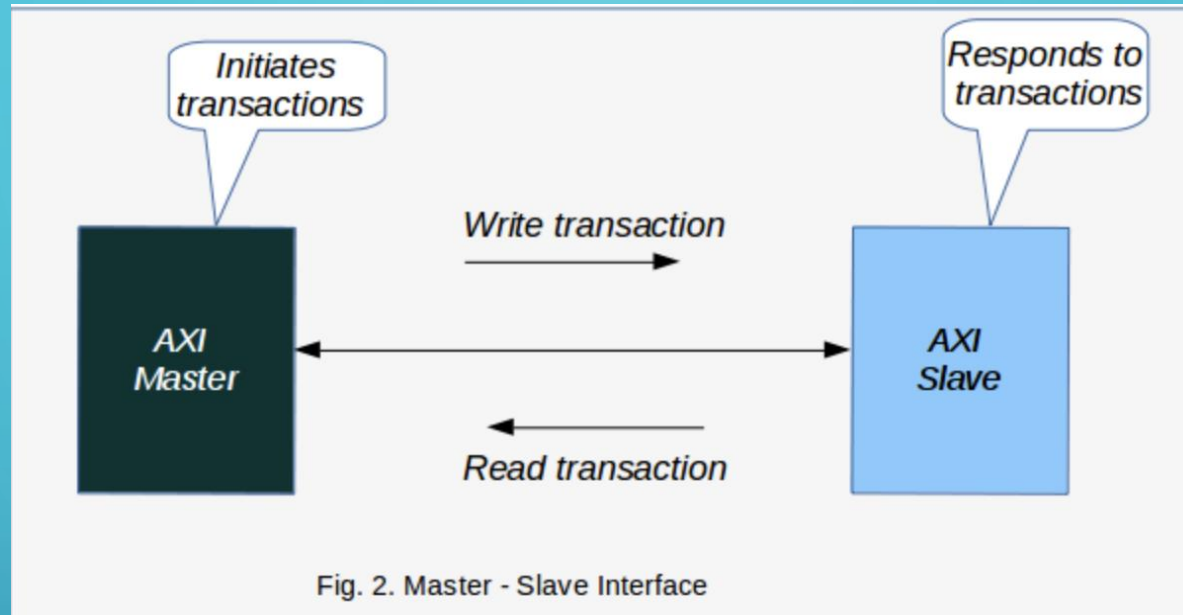
```
static inline void keccakf(void* state) {
    uint64_t* a = (uint64_t*)state;
    uint64_t b[5] = {0};
    uint64_t t = 0;
    uint8_t x, y, i = 0;

    REPEAT24(
        // Theta
        FOR5(x, 1,
            b[x] = 0;
            FOR5(y, 5,
                b[x] ^= a[x + y]; ))
        FOR5(x, 1,
            FOR5(y, 5,
                a[y + x] ^= b[(x + 4) % 5] ^ rol(b[(x + 1) % 5], 1); ))
        // Rho and pi
        t = a[1];
        x = 0;
        REPEAT24(b[0] = a[pi[x]];
            a[pi[x]] = rol(t, rho[x]);
            t = b[0];
            x++; )
        // Chi
        FOR5(y,
            5,
            FOR5(x, 1,
                b[x] = a[y + x];)
            FOR5(x, 1,
                a[y + x] = b[x] ^ ((~b[(x + 1) % 5]) & b[(x + 2) % 5]); ))
        // Iota
        a[0] ^= RC[i];
        i++; )
    }
```

AXI-LITE

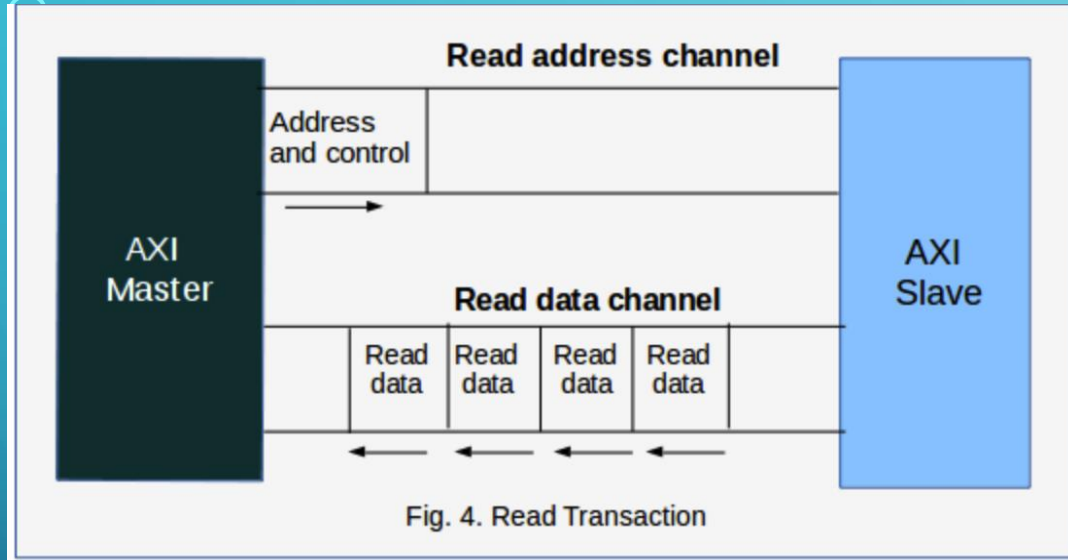
- Simplified version of the full AXI Protocol used for register level communication with single beat memory mapped transfers.
- Typically 32-bits wide, but we are testing with 64-bits as well by changing the `AXI_DATA_WIDTH`.
- Why AXI Lite ?
 - We are only sending small commands of data, such as the nonce, so Lite is sufficient.
 - Easy to intergrate with Zync PS.
 - Keeps hardware design clean and allows predicatble timing.

HOW AXI LITE WORKS ?



- AXI-Lite is based on a master-slave architecture, where AXI Master (PS) initiates the read/write transactions and AXI Slave (Custom Peripheral) responds to those.
- Transactions are unidirectional with the handshake protocol.
- AXI-Lite enables memory-mapped I/O by assigning each peripheral register in the slave a fixed address in the processor's memory space, allowing the PS to access hardware registers using standard load/store instructions.

HOW AXI LITE WORKS ?



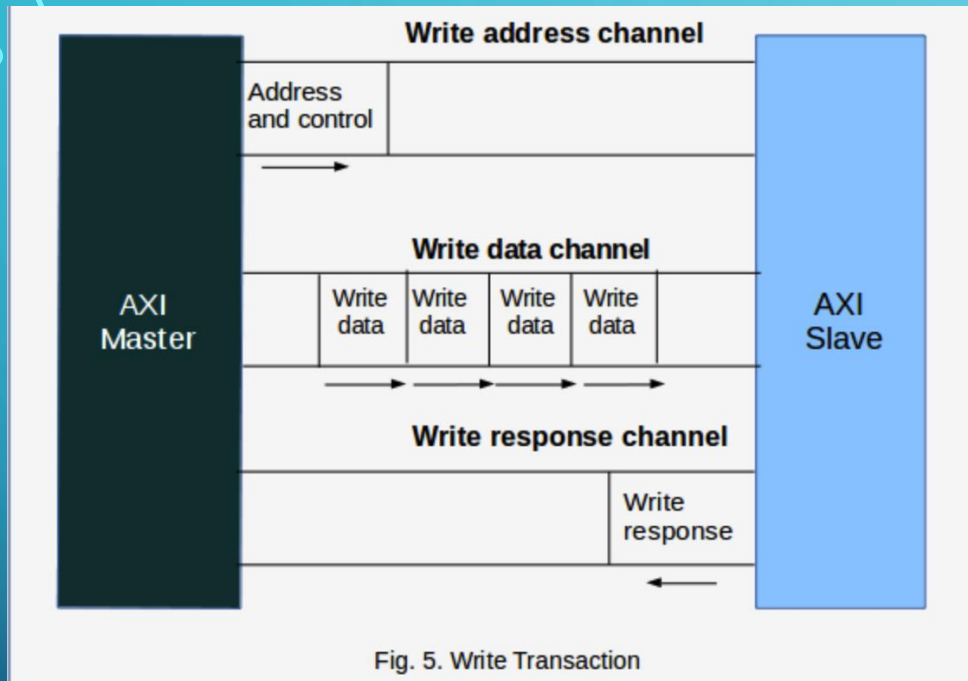
- **AXI4-Lite transactions use five separate channels:**

- Read Address, Write Address, Read Data, Write Data, and Write Response — each with its own handshaking signals for independent and reliable communication.

- **Read Transaction:**

- The master initiates a read by sending an address over the **read address channel**; the slave responds with data over the **read data channel**, using handshakes to synchronize the transfer.

HOW AXI LITE WORKS ?



Write Transaction:

- The master sends the target address via the **write address channel**, followed by data on the **write data channel**. The slave responds with an acknowledgment on the **write response channel**, confirming success or error.

Handshake Protocol:

- All channels use VALID/READY handshake to coordinate the data transfer. Sender asserts VALID when there's data to be sent and receiver asserts READY when it is prepared to accept data. The transfer only happens when both signals are high on same clock edge.
- This ensures reliable and synchronized communication between PS and PL.

SOFTWARE ARCHITECTURE (PS-MASTER)

```
    return -1;
}

// Set input signals to AXI registers
uint32_t nonce_start = 0x00000000;
REG(REG_NONCE_START) = nonce_start;
REG(REG_TARGET_LO) = 0x000000FF;
REG(REG_TARGET_HI) = 0x00000000;

// Start the miner - set only 1 control bit
REG(REG_CTRL_READY) = 0x01;

uint32_t compare_data = 0;
uint32_t compare_count = 0;
bool valid_found = false;

// Polling
printf("Waiting for valid nonce to be found or max limit to be reached\n");

while (1) { // checks compare count continuously
    compare_data = REG(REG_COMPARE_COUNT);

    // Check is valid bit is set to 1
    valid_found = (compare_data >> 31) & 0x1;

    // Get compare count
    compare_count = compare_data & COUNT_MASK;

    // Exit loop is nonce found or compare limit reached
    if (valid_found || compare_count >= MAX_COMPARE_COUNT) {
        uint32_t valid_nonce = nonce_start + compare_count;

        if (valid_found) {
            printf("Valid nonce: 0x%08X\n", valid_nonce);
        } else {
            printf("Max compare limit reached, no valid nonce found.\n");
        }

        break;
    }
}
```

Initial Setup:

- Set the NONCE_START, TARGET_LO, and TARGET_HI registers with starting values.
- Trigger the miner by writing 0x01 to the CTRL register (start bit).

Polling Loop:

- Continuously reads from COMPARE_COUNT register.
- Bit 31 is used as a **valid flag** — set when a valid nonce is found.
- Lower 31 bits represent the **current compare count**.

Exit Conditions:

- Breaks when:
 - A valid nonce is found (valid_found == 1), or
 - Maximum comparisons reached (compare_count >= MAX_COMPARE_COUNT).

Result Handling:

- If valid, prints the nonce.
- Else, prints that max attempts were reached.
- Adds 1ms delay per iteration to avoid excessive CPU usage.

AXI SLAVE PERIPHERAL DESIGN

```
-- memory maps - data from PS (AXI controller in PL)
-- TODO debug
process (S_AXI_ACLK)
variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
begin
    if rising_edge(S_AXI_ACLK) then
        if S_AXI_ARESETN = '0' then
            n_reg_we    <= '0';
            n_reg_din   <= (others => '0');
            t_reg_we    <= '0';
            t_reg_din   <= (others => '0');
            init_reg_we <= '0';
            init_reg    <= '0';
        else
            n_reg_we    <= '0';
            t_reg_we    <= '0';
            init_reg_we <= '0';
            loc_addr := axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
            if (slv_reg_wren = '1') then
                case loc_addr is
                    when "00" => -- nonce
                        n_reg_din <= S_AXI_WDATA;
                        n_reg_we  <= '1';
                    when "01" => -- target
                        t_reg_din <= S_AXI_WDATA;
                        t_reg_we  <= '1';
                    when "10" => -- init/start
                        init_reg <= S_AXI_WDATA(0);
                        init_reg_we <= '1';
                    when others =>
                        null;
                end case;
            end if;
        end if;
    end if;
end process;
```

Custom module in the hardware that allows the PS to communicate with PL via memory-mapped transactions.

1. Register Write Handling

- PS writes nonce, target, and start signal to AXI registers
- Address decoding logic routes each write to the correct register

2. Keccak Control Integration

- Controller module is triggered via `init_reg`
- Receives nonce and target directly from AXI-mapped registers

3. Register Readback

- Resulting `found_nonce` is stored and made accessible to the PS
- PS reads it via a standard AXI read transaction

```
void test_speed()
{
    int i;
    uint64_t st[25], x, n;
    clock_t bg, us;

    for (i = 0; i < 25; i++)
        st[i] = i;

    bg = clock();
    n = 0;
    do {
        for (i = 0; i < 100000; i++)
            sha3_keccakf(st);
        n += i;
        us = clock() - bg;
    } while (us < 3 * CLOCKS_PER_SEC);

    x = 0;
    for (i = 0; i < 25; i++)
        x += st[i];

    printf("(%016lx) %.3f Keccak-p[1600,24] / Second.\n",
        (unsigned long) x, (CLOCKS_PER_SEC * ((double) n))
```

BENCHMARK METHODOLOGY

- Sustained computation (mining cycle)
- Burst compute for 1M hash
 - Factors in AXI latency
- Power monitoring
 - With PowerStat and XPE

PERFORMANCE EVALUATION: KECCAK

	SUSTAINED WRITE (MH/S)	TIME FOR 1M HASH	POWER USAGE (W)
Sha-3 Miner (VHDL)	-	-	2.141
Tiny-sha3 (HLS)	6.281	0.256s	2.422
Tiny-sha3 (CPU: ARM A53 (KV260))	0.828	1.315s	4.155
Tiny-sha3 (CPU: R9 5900HS)	1.595	0.611s	12.54

CURRENT CONCLUSIONS

The acceleration effect towards SHA-3 provided by HLS is pronounced. If more time are to be given for our VHDL implementation, our implementation with VHDL would likely supercede it both performance and efficiency-wise.

ROLES OF TEAM MEMBERS

- Felix: Working on Keccak Core Design, benchmarking, and hash computing
- Desmond: Debugging core VHDL, Top Level entity for bitstream generation, constraints for core algorithm
- Jack: Working on system integration, PS-PL coordination and hardware testing
- Thenuja: Contributing to optimization strategy, PS-PL coordination and documentation



THANK YOU